Design and Implementation of a Defensive Malware Scanning System Using C++ and Object-Oriented Programming

Emirhan
Dept. of Computer Engineering
Kastamonu University
Kastamonu, Turkey
[E-posta Adresin]

Abstract—This paper presents the design and implementation of a defensive malware scanning system developed in C++ using Object-Oriented Programming (OOP) principles. The system is built on a modular architecture, leveraging polymorphism and encapsulation techniques. It features a dual-layer analysis mechanism consisting of a signature-based engine that checks for known malware patterns and a heuristic engine that analyzes suspicious command sequences within files. Experimental results demonstrate that the system successfully identifies malicious files and reports analysis findings into a persistent log file.

Keywords—malware analysis, object-oriented programming, antivirus, C++, cyber security

I. Introduction

The increasing complexity and volume of cyber threats in the modern era necessitate the adoption of robust defensive software development practices to ensure system security. Malware analysis stands as one of the most fundamental steps in protecting endpoint systems. In this context, scanning engines capable of examining file contents and behavioral characteristics form the core of antivirus systems.
The primary objective of this project is to develop an extensible defensive file scanning system by combining the low-level memory management advantages of the C++ programming language with the modularity provided by the Object-Oriented Programming (OOP) architecture. The software is designed with a dual-layer structure that is not limited to traditional methods of checking known threat databases but is also capable of analyzing suspicious command calls.

II. System Architecture and Methodology

The developed scanning system is divided into modular classes that work integrated with each

other, adhering to the Single Responsibility Principle. The architecture consists of a file management layer, polymorphic analysis engines, and a central reporting unit.

A. File Management and Encapsulation

The input mechanism of the system is managed by the `FileAnalyzer` class. This class is responsible for safely reading target files in binary format without causing memory leaks. File paths and content data within the class are restricted from direct external access through the principle of encapsulation and are only accessible through controlled methods.

B. Polymorphism and Scanning Engines

The design of the analysis engines is built upon an abstract interface named `IScanner` to ensure the system remains open to new scanning techniques in the future. Utilizing the capability of polymorphism through this interface, two different security layers have been created:

- *Heuristic Scanner:* Dynamically analyzes suspicious system commands, script sequences, or memory manipulation calls (e.g., system(), eval()) found within the file content that carry potential threats.
- *Signature Scanner:* Integrated with the `SignatureDatabase` class, it matches scanned file contents and hashes against known malware signatures within milliseconds to catch definitive threats.

C. Central Analysis Engine and Reporting

The management of all scanner modules and file routing within the system is provided by a central engine called `AntivirusEngine`. Incorporating the scanners through the composition method, this engine triggers the `ReportGenerator` class upon completion of the analysis process to record transaction results, detected threats, and timestamps into a persistent log file.

III. Test and Results

To measure the accuracy, detection capability, and error margin of the developed system, a controlled analysis simulation was conducted using three test files with different characteristics. The tests were performed on a completely clean text file, a script file containing suspicious operating system commands, and a dummy virus file containing a known malware signature.

- *Clean File Analysis:* When a file containing only standard text data was loaded into the system, both scanning engines examined the file and marked it as "Safe" without producing any false-positive warnings.
- *Heuristic Detection:* When a suspicious .bat file containing operating system-level calls such as system() was analyzed, the Heuristic Engine detected the potential for abnormal behavior in the file without the need for a signature database and halted the process.
- *Signature-Based Detection:* A file harboring a pre-defined malicious hash/signature value in the database was scanned and immediately blocked as a definitive threat by the Signature Scanner.

As a result of the tests, it was observed that the central `AntivirusEngine` successfully

coordinated all processes and recorded the obtained results completely in the `scan_report.txt` log file, including the timestamp, the scanned file path, and the identifying engine information.

IV. Conclusion

In this study, a prototype of a defensive malware scanning system for endpoint security was successfully designed and developed using C++. The integration of Object-Oriented Programming (OOP) principles (encapsulation, inheritance, polymorphism) into the core architecture of the project provided the system with a high level of modularity and sustainability. Managing the scanning engines through a polymorphic interface allows the software to incorporate more advanced engines, such as machine learning-based behavioral analysis, in the future. Consequently, the developed system has presented the basic operating principles of modern antivirus software and software engineering standards within an academic framework.

References

1. B. Stroustrup, *The C++ Programming Language*, 4th ed., Boston, MA, USA: Addison-Wesley, 2013.
2. M. Sikorski and A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*, San Francisco, CA, USA: No Starch Press, 2012.
3. E. Gandotra, D. Bansal, and S. Sofat, "Malware Analysis and Classification: A Survey," *Journal of Information Security*, vol. 5, pp. 56-64, 2014.
4. R. C. Seacord, *Secure Coding in C and C++*, 2nd ed., Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2013.